```
from math import prod

import numpy as np
import pandas as pd
from IPython.display import display

x_star = 3
y = lambda x: np.sqrt(x)
x_a = np.array([0, 1.7, 3.4, 5.1])
x_b = np.array([0, 1.7, 4, 5.1])
eps = 1e-3
n = len(x_a)
```

```
y_a = y(x_a)
y_b = y(x_b)
display(pd.DataFrame({"x": x_a, "y": y_a}), pd.DataFrame({"x": x_b, "y": y_b}))
```

	x	y
0	0.0	0.000000
1	1.7	1.303840
2	3.4	1.843909
3	5.1	2.258318

	x	y
0	0.0	0.000000
1	1.7	1.303840
2	4.0	2.000000
3	5.1	2.258318

Интерполяция Лагранжа

```
import sympy as sp

x = sp.Symbol('x')

L_a = sum(y_a[i] * prod((x - x_a[j]) / (x_a[i] - x_a[j]) for j in range(n) if j != i)
L_b = sum(y_b[i] * prod((x - x_b[j]) / (x_b[i] - x_b[j]) for j in range(n) if j != i)
```

```
derivative = sp.diff(sp.sqrt(x), x, n)

derivative
```

$$-\frac{15}{16x^{\frac{7}{2}}}$$

Максимум достигается в точке $x = 0$, так как функция убывает, а её производные положительны. Посчитать M не получится...

```
# Ma = max(float(derivative.subs(x, x_a[i]).evalf()) for i in range(n))
# Mb = max(float(derivative.subs(x, x_b[i]).evalf()) for i in range(n))
#
# w_a = prod(x - i for i in x_a)
# w_b = prod(x - i for i in x_b)
#
# error_a = Ma * abs(w_a) / factorial(n)
# error_b = Mb * abs(w_b) / factorial(n)
#
# error_a = float(error_a.subs(x, x_star).evalf())
# error_b = float(error_b.subs(x, x_star).evalf())
#
dif_a_lagrange = abs(y(x_star) - float(L_a.subs(x, x_star).evalf()))
dif_b_lagrange = abs(y(x_star) - float(L_b.subs(x, x_star).evalf()))
#
# dif_a <= error_a, dif_b <= error_b
```

```
dif_a_lagrange, dif_b_lagrange
```

```
(np.float64(0.01972677181660165), np.float64(0.04266258595181016))
```

Интерполяция Ньютона

```
from functools import lru_cache

@lru_cache(maxsize=None)
def divided_diff(f, *x_args):
    if len(x_args) == 1:
        return f(x_args[0])
    else:
        return (divided_diff(f, *x_args[1:]) - divided_diff(f, *x_args[:-1])) / (x_ar
```

```
def get_w(n, x_data):
    return prod(x - i for i in x_data[:n])

P_newton_a = sum(get_w(i, x_a) * divided_diff(y, *x_a[:i + 1]) for i in range(n))
P_newton_b = sum(get_w(i, x_b) * divided_diff(y, *x_b[:i + 1]) for i in range(n))
```

```
dif_a_newton = abs(y(x_star) - float(P_newton_a.subs(x, x_star).evalf()))
dif_b_newton = abs(y(x_star) - float(P_newton_b.subs(x, x_star).evalf()))
#
# dif_a <= error_a, dif_b <= error_b
```

```
dif_a_newton, dif_b_newton
```

```
(np.float64(0.019726771816601874), np.float64(0.04266258595180972))
```

Валидация

```
pts_a = [(sp.nsimplify(xi), sp.sqrt(sp.nsimplify(xi))) for xi in x_a]
a_validate = sp.expand(sp.interpolate(pts_a, x))

display(
    sp.expand(L_a),
    sp.N(sp.expand(a_validate), 15)
)
np.isclose(
    float(L_a.subs(x, x_star).evalf()),
    float(a_validate.subs(x, x_star).evalf()),
    eps
)
```

$0.0216470834816846x^3 - 0.24254062240408x^2 + 1.11672397567224x$

$0.0216470834816847x^3 - 0.24254062240408x^2 + 1.11672397567224x$

```
np.True_
```

```
pts_b = [(sp.nsimplify(xi), sp.sqrt(sp.nsimplify(xi))) for xi in x_b]
b_validate = sp.expand(sp.interpolate(pts_b, x))

display(
    sp.expand(L_b),
    sp.N(sp.expand(b_validate), 16)
)
np.isclose(
    float(L_b.subs(x, x_star).evalf()),
    float(b_validate.subs(x, x_star).evalf()),
    eps
)
```

$0.0188466177753101x^3 - 0.223497455600733x^2 + 1.09244393799797x$

$0.01884661777531005x^3 - 0.2234974556007327x^2 + 1.09244393799797x$

```
np.True_
```

```
display(
    sp.expand(P_newton_a),
    sp.N(sp.expand(a_validate), 15)
)
np.isclose(
    float(P_newton_a.subs(x, x_star).evalf()),
    float(a_validate.subs(x, x_star).evalf()),
    eps
)
```

$0.0216470834816846x^3 - 0.24254062240408x^2 + 1.11672397567224x$

$0.0216470834816847x^3 - 0.24254062240408x^2 + 1.11672397567224x$

```
np.True_
```

```
display(
    sp.expand(P_newton_b),
    sp.N(sp.expand(b_validate), 15)
)
np.isclose(
    float(P_newton_b.subs(x, x_star).evalf()),
    float(b_validate.subs(x, x_star).evalf()),
    eps
)
```

$0.01884661777531x^3 - 0.223497455600733x^2 + 1.09244393799797x$

$0.01884661777531x^3 - 0.223497455600733x^2 + 1.09244393799797x$

```
np.True_
```